Nettoyage des données · Binôme 08 · Catalogue Netflix

AMA PIIA cohorte 2 · exercice en binômes du 30/10/2026 **Binôme** : QUENUM Amen et AFOUDA Omotola · **Domaine** : Médias

Sujet : nettoyer le catalogue des films et séries de Netflix pour analyser les contenus par pays, genre et année.

Données : https://www.kaggle.com/datasets/shivamb/netflix-shows (fichier `netflix_titles.csv`, copie intacte dans `data/raw/`)

Problèmes annoncés (liste non exhaustive, en trouver d'autres fait partie de l'exercice) : réalisateurs et acteurs manquants, dates d'ajout en texte, durées mélangées (minutes et saisons), colonnes à plusieurs valeurs.

Partie	Contenu	Qui
1	État des lieux avant correction	à deux
2.1 à 2.4	Texte, décalages, dates, durées	Omotola
2.5 à 2.7	Manquants, colonnes à plusieurs valeurs, autres problèmes	Amen
3	État final	à deux
4	Export du CSV nettoyé	à deux

Règle du binôme : chaque correction a une cellule de code **et** une cellule « Décision » qui dit ce qui a été fait, pourquoi, et combien de lignes sont touchées.

```
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# On remonte jusqu'au dossier qui contient requirements.txt : comme ça les chemins marchent chez nous deux
RACINE = next(p for p in [Path.cwd(), *Path.cwd().parents] if (p / "requirements.txt").exists())
BRUT = RACINE / "data" / "raw" / "netflix_titles.csv"
PROPRE = RACINE / "data" / "processed" / "netflix_titles_propre.csv"

pd.set_option("display.max_columns", None)  # sinon pandas cache les colonnes du milieu
```

1. État des lieux (avant toute correction)

On charge le fichier brut **une seule fois**, puis on travaille sur une copie : `brut` ne doit plus jamais être modifié. L'état des lieux se lit sur `brut`, les corrections de la partie 2 se font sur `df`. Comme ça, les chiffres de cette partie restent ceux du fichier d'origine, même si on relance une cellule après avoir nettoyé.

```
brut = pd.read_csv(BRUT)
df = brut.copy()
df.head()
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	As her father nears the end of his life, filmm...
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, TV Dramas, TV Mysteries	After crossing paths at a party, a Cape Town t...
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	September 24, 2021	2021	TV-MA	1 Season	Crime TV Shows, International TV Shows, TV Act...	To protect his family from a powerful drug lor...
3	s4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	September 24, 2021	2021	TV-MA	1 Season	Docuseries, Reality TV	Feuds, flirtations and toilet talk go down amo...
4	s5	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, Romantic TV Shows, TV ...	In a city of coaching centers known to train I...

1.1 Dimensions et types

Combien de lignes et de colonnes contient le fichier, et chaque colonne a-t-elle un type qui correspond à son contenu ?

```
print(f"{brut.shape[0]} lignes, {brut.shape[1]} colonnes")

# Pour chaque colonne : comment pandas l'a lue, et combien de valeurs différentes elle contient
pd.DataFrame({
    "type" : brut.dtypes,
    "valeurs_differentes" : brut.nunique(),
})
```

```
8807 lignes, 12 colonnes
```

	type	valeurs_differentes
show_id	str	8807
type	str	2
title	str	8807
director	str	4528
cast	str	7692
country	str	748
date_added	str	1767
release_year	int64	74
rating	str	17
duration	str	220
listed_in	str	514
description	str	8775

Constats

Le fichier compte 8 807 lignes et 12 colonnes. Chaque ligne est un film ou une série, et `show_id` (l'identifiant de chaque titre) est bien unique : 8 807 valeurs différentes pour 8 807 lignes.

La colonne « type » du tableau indique comment pandas a lu chaque colonne. Seule `release_year` (l'année de sortie) est lue comme un nombre (`int64`), tout le reste est lu comme du texte (`str`). C'est normal pour un titre ou une description, mais pas pour deux colonnes. `date_added` (la date d'ajout sur Netflix) contient des dates écrites en toutes lettres, comme « September 25, 2021 ». Tant que c'est du texte, l'ordinateur ne sait pas que c'est une date, donc impossible de trier par date ou de compter les ajouts par année. On la convertit en 2.3. Quant à `duration` (la durée), elle mélange des minutes pour les films (« 90 min ») et des saisons pour les séries (« 2 Seasons »), et on ne peut faire aucun calcul dessus. Il faudra la couper en deux colonnes de nombres (2.4).

On remarque aussi que `rating` (la classification par âge, par exemple « TV-MA » pour les adultes) n'a que 17 valeurs différentes. C'est assez peu pour toutes les regarder une par une, ce qu'on fait en 1.4.

1.2 Valeurs manquantes

Pour chaque colonne, on regarde le nombre **et** le pourcentage de cases vides. Avant de corriger quoi que ce soit, il faut comprendre *pourquoi* une valeur manque : un oubli, une information qui n'existe pas, ou une erreur de saisie ?

```
# On en fait une fonction pour pouvoir la relancer sur df en partie 3 (avant / après)
def manquants(d):
    """Nombre et pourcentage de valeurs manquantes par colonne."""
    return pd.DataFrame({
        "manquants": d.isna().sum(),
        "pct" : (d.isna().mean() * 100).round(1),
    }).sort_values("manquants", ascending=False)

manquants(brut)
```

	manquants	pct
director	2634	29.9
country	831	9.4
cast	825	9.4
date_added	10	0.1
rating	4	0.0
duration	3	0.0
show_id	0	0.0
type	0	0.0
title	0	0.0
release_year	0	0.0
listed_in	0	0.0
description	0	0.0

```
# Presque 30 % de réalisateurs manquants, ça nous a paru énorme.
# On regarde si c'est pareil pour les films et pour les séries (en % de chaque type)
pd.crosstab(brut["type"], brut["director"].isna(), normalize="index").round(3) * 100
```

director type	False	True
Movie	96.9	3.1
TV Show	8.6	91.4

Constats

Six colonnes ont des valeurs manquantes (des cases vides, que pandas affiche `NaN`). Trois comptent vraiment : `director` (le réalisateur, 29,9 % de vides), `country` (le pays, 9,4 %) et `cast` (les acteurs, 9,4 %). Pour les autres c'est très peu, avec 10 dates d'ajout, 4 classifications et 3 durées.

Le cas de `director` nous a surpris au départ : presque un tiers du catalogue sans réalisateur. Le tableau croisé (un tableau qui compare deux colonnes, ici le type de contenu et le fait que le réalisateur soit vide ou non) explique tout. Le réalisateur manque pour 91,4 % des séries mais seulement pour 3,1 % des films. Une série est souvent tournée par plusieurs réalisateurs, parfois un par épisode, donc la case vide ne veut pas dire qu'on a perdu l'information. Supprimer ces lignes reviendrait à jeter presque toutes les séries, et remplir la case serait inventer. On en discute en 2.5.

Les 3 durées manquantes ne le sont pas vraiment : on les retrouve dans `rating` (voir 1.4).

1.3 Doublons

On cherche à deux niveaux : des lignes strictement identiques, et un même contenu présent deux fois avec un identifiant différent.

```
print("Doublons stricts :", brut.duplicated().sum())
print("show_id en double :", brut["show_id"].duplicated().sum())

# Rien de ce côté-là, mais un même titre peut s'écrire avec des majuscules différentes.
# On compare donc les titres en minuscules et sans les espaces autour
titre_normalise = brut["title"].str.lower().str.strip()
print("Titres en double (casse et espaces ignorés) :", titre_normalise.duplicated().sum())

brut[titre_normalise.duplicated(keep=False)].sort_values("title")[
    ["show_id", "type", "title", "director", "release_year"]
]
```

```
Doublons stricts : 0
show_id en double : 0
Titres en double (casse et espaces ignorés) : 6
```

	show_id	type	title	director	release_year
3371	s3372	Movie	Consequences	Ozan Açıktan	2014
6529	s6530	Movie	Consequences	Ozan Açıktan	2014
5751	s5752	TV Show	DEATH NOTE	NaN	2006
5318	s5319	Movie	Death Note	Adam Wingard	2017
6705	s6706	Movie	Esperando La Carroza	Alejandro Doria	1985
303	s304	Movie	Esperando la carroza	Alejandro Doria	1985
5033	s5034	Movie	FullMetal Alchemist	Fumihiko Sori	2017
5095	s5096	TV Show	Fullmetal Alchemist	NaN	2003
7345	s7346	Movie	Love In A Puff	Pang Ho-cheung	2010
159	s160	Movie	Love in a Puff	Pang Ho-cheung	2010
8022	s8023	TV Show	Sin Senos sí Hay Paraíso	NaN	2018
1270	s1271	TV Show	Sin senos sí hay paraíso	NaN	2018

Constats

Un doublon, c'est une même ligne présente deux fois dans le fichier. Aucune ligne n'est copiée à l'identique et aucun `show_id` n'apparaît deux fois. On aurait pu s'arrêter là, mais on a aussi comparé les titres en ignorant les majuscules et les espaces en trop, et 6 titres sortent en double. On les a regardés un par un.

Death Note et *Fullmetal Alchemist* existent chacun en film récent et en série d'animation japonaise plus ancienne. Ce sont deux œuvres différentes qui portent le même nom, on garde les deux.

Les deux *Consequences* sont identiques sur toutes les colonnes. Le second titre se termine par un caractère invisible (un espace insécable, voir 1.4), et c'est pour ça que pandas ne les voyait pas comme des doublons.

Esperando la carroza, *Love in a Puff* et *Sin senos sí hay paraíso* ont le même réalisateur, la même année et la même description, mais deux dates d'ajout différentes (2018 ou 2019, puis 2021). Netflix a probablement retiré ces titres puis les a remis en ligne.

Il y a donc 4 vrais doublons sur 6 paires. Ce qu'on en fait est traité en 2.7.

1.4 Valeurs suspectes et aberrantes

On passe en revue ce qui peut cacher une erreur : les valeurs des colonnes courtes (`type` , `rating`), les unités de `duration` , les années de `release_year` , le format de `date_added` , puis les virgules et les caractères invisibles dans le texte. Une valeur qui n'a rien à faire dans sa colonne est un indice de **décalage**.

```
# type et rating ont peu de valeurs, on peut les afficher en entier
print(brut["type"].value_counts(dropna=False), "\n")
print(brut["rating"].value_counts(dropna=False))
```

```
type
Movie      6131
TV Show    2676
Name: count, dtype: int64

rating
TV-MA       3207
TV-14       2160
TV-PG        863
R            799
PG-13        490
TV-Y7        334
TV-Y         307
PG           287
TV-G         220
NR            80
G             41
TV-Y7-FV       6
NaN            4
NC-17          3
UR             3
74 min         1
84 min         1
66 min         1
Name: count, dtype: int64
```

```
# "74 min" dans rating ?! On regarde ces lignes de plus près
decalees = brut["rating"].str.contains("min", na=False)
brut.loc[decalees, ["show_id", "title", "director", "rating", "duration"]]
```

	show_id	title	director	rating	duration
5541	s5542	Louis C.K. 2017	Louis C.K.	74 min	NaN
5794	s5795	Louis C.K.: Hilarious	Louis C.K.	84 min	NaN
5813	s5814	Louis C.K.: Live at the Comedy Store	Louis C.K.	66 min	NaN

```
# Est-ce que les films sont toujours en minutes et les séries toujours en saisons ?
unite = brut["duration"].str.extract(r"^\d+ (\w+)$")[0]
pd.crosstab(brut["type"], unite.fillna("vide"))
```

type	Season	Seasons	min	vide
Movie	0	0	6128	3
TV Show	1793	883	0	0

```
# Années de sortie : on cherche des valeurs impossibles (trop anciennes, dans le futur...)
print(brut["release_year"].describe())
brut.nsmallest(3, "release_year")[["title", "type", "release_year"]]
```

```
count    8807.000000
mean     2014.180198
std         8.819312
min      1925.000000
25%      2013.000000
50%      2017.000000
75%      2019.000000
max      2021.000000
Name: release_year, dtype: float64
```

	title	type	release_year
4250	Pioneers: First Women Filmmakers*	TV Show	1925
7790	Prelude to War	Movie	1942
8205	The Battle of Midway	Movie	1942

```
print("Dates qui commencent par un espace :", brut["date_added"].str.startswith(" ").sum())

# Simple essai de conversion pour voir si ça passe, la vraie conversion se fait en 2.3 sur df
essai = pd.to_datetime(brut["date_added"].str.strip(), format="%B %d, %Y", errors="coerce")
```

```
print("Dates non convertibles :", essai.isna().sum(), "dont", brut["date_added"].isna().sum(), "déjà vides")
print("Période couverte :", essai.min().date(), "à", essai.max().date())

# Un titre ajouté sur Netflix avant d'être sorti, normalement c'est impossible
avant_sortie = essai.dt.year < brut["release_year"]
print("Ajoutés avant leur année de sortie :", avant_sortie.sum())
brut.loc[avant_sortie, ["title", "type", "release_year", "date_added"]]
```

```
Dates qui commencent par un espace : 88
Dates non convertibles : 10 dont 10 déjà vides
Période couverte : 2008-01-01 à 2021-09-25
Ajoutés avant leur année de sortie : 14
```

	title	type	release_year	date_added
1551	Hilda	TV Show	2021	December 14, 2020
1696	Polly Pocket	TV Show	2021	November 15, 2020
2920	Love Is Blind	TV Show	2021	February 13, 2020
3168	Fuller House	TV Show	2020	December 6, 2019
3287	Maradona in Mexico	TV Show	2020	November 13, 2019
3369	BoJack Horseman	TV Show	2020	October 25, 2019
3433	The Hook Up Plan	TV Show	2020	October 11, 2019
4844	Unbreakable Kimmy Schmidt	TV Show	2019	May 30, 2018
4845	Arrested Development	TV Show	2019	May 29, 2018
5394	Hans Teeuwen: Real Rancour	Movie	2018	July 1, 2017
5658	Sense8	TV Show	2018	December 23, 2016
5677	Tokyo Trial	TV Show	2017	December 13, 2016
7063	Incoming	Movie	2019	October 26, 2018
7112	Jack Taylor	TV Show	2016	March 31, 2013

```
# Combien de cases contiennent plusieurs valeurs séparées par une virgule ?
multi = ["director", "cast", "country", "listed_in"]
print(brut[multi].apply(lambda s: s.str.contains(",", na=False).sum()), "\n")

# En parcourant country on est tombés sur des virgules au début ou à la fin
mal_placees = brut["country"].str.startswith(",", na=False) | brut["country"].str.endswith(",", na=False)
brut.loc[mal_placees, ["show_id", "title", "country"]]
```

```
director      614
cast         7101
country      1320
listed_in    6787
dtype: int64
```

	show_id	title	country
193	s194	D.P.	, South Korea
365	s366	Eyes of a Thief	, France, Algeria
1192	s1193	The Present	United Kingdom,
2224	s2225	Funan	France, Belgium, Luxembourg, Cambodia,
4653	s4654	City of Joy	United States,
5925	s5926	Virunga	United Kingdom,
7007	s7008	Hotel Pacific	Poland,

```
# Caractères invisibles : on les cherche un par un dans chaque colonne texte
# (\xa0 = espace insécable, \u200b = espace de largeur nulle)
texte = brut.select_dtypes(exclude="number").columns
pd.DataFrame({
    "espace_debut_fin": {c: (brut[c].notna() & (brut[c].str.strip() != brut[c])).sum() for c in texte},
    "espace_insecable": {c: brut[c].str.contains("\xa0", regex=False, na=False).sum() for c in texte},
    "largeur_nulle":    {c: brut[c].str.contains("\u200b", regex=False, na=False).sum() for c in texte},
    "double_espace":    {c: brut[c].str.contains("  ", regex=False, na=False).sum() for c in texte},
})
```

	espace_debut_fin	espace_insecable	largeur_nulle	double_espace
show_id	0	0	0	0
type	0	0	0	0
title	1	5	5	3
director	0	0	0	0
cast	0	0	1	7
country	0	0	0	0
date_added	88	0	0	0
rating	0	0	0	0
duration	0	0	0	0
listed_in	0	0	0	0
description	0	9	0	6

Constats

`type` ne pose aucun problème : deux valeurs, 6 131 films (`Movie`) et 2 676 séries (`TV Show`).

`rating` est plus intéressante. Trois lignes contiennent « 74 min », « 84 min » et « 66 min », ce qui n'est évidemment pas une classification par âge. Ce sont trois spectacles de Louis C.K., et leur `duration` est vide : la durée a été saisie une colonne trop à gauche. C'est ce qu'on appelle une valeur décalée, et on la remet à sa place en 2.2. On note aussi que `NR` (*Not Rated*, « non classé », 80 titres) et `UR` (*Unrated*, « sans classification », 3 titres) veulent dire la même chose, et que `TV-Y7-FV` (6 titres) n'est que `TV-Y7` (pour les plus de 7 ans) avec une mention « violence fantastique » en plus. Ces deux points sont discutés en 2.7.

Pour `duration` , le tableau croisé est rassurant. Les films sont tous en minutes et les séries toutes en saisons, sans aucune exception. Les seuls vides sont les trois spectacles de Louis C.K., donc la séparation en deux colonnes (2.4) devrait se faire sans rien perdre.

On s'est d'abord méfiés de la plus petite année de sortie, 1925, qui pouvait être une faute de frappe. Vérification faite, *Pioneers: First Women Filmmakers* est une collection de films muets tournés par des réalisatrices dans les années 1920, et les titres de 1942 sont des documentaires sur la Seconde Guerre mondiale. Les années vont de 1925 à 2021 et rien n'est aberrant (aucune valeur impossible ou absurde).

`date_added` se convertit en date sans difficulté une fois retirés les espaces en trop au début (88 dates concernées). Aucune conversion n'échoue en dehors des 10 cases déjà vides, et les dates vont de 2008 à 2021. En comparant avec `release_year` , on trouve en revanche 14 titres ajoutés sur Netflix avant leur année de sortie, ce qui paraît impossible. Douze sont des séries, comme *BoJack Horseman* ou *Arrested Development*. Notre hypothèse : pour une série, `release_year` indique l'année de la dernière saison, pas celle de la première. On garde ces lignes telles quelles et on l'explique en 2.7.

Le texte cache aussi des caractères invisibles. Ils ne se voient pas à l'écran, mais pour Python deux textes qui ont l'air identiques deviennent différents :

- des espaces insécables (`\xa0` , une espace spéciale qui empêche le retour à la ligne) dans 5 titres et 9 descriptions, dont le *Consequences* vu en 1.3 ;
- des espaces de largeur nulle (`\u200b` , un caractère qui ne prend aucune place à l'écran) dans 5 titres et 1 liste d'acteurs. Ceux-là sont totalement invisibles, même quand on affiche la valeur ;
- des doubles espaces dans 3 titres, 7 listes d'acteurs et 6 descriptions.

Tout cela est nettoyé en 2.1.

Pour finir, quatre colonnes contiennent souvent plusieurs valeurs dans une même case, séparées par des virgules : `cast` (7 101 lignes), `listed_in` (les genres, 6 787), `country` (1 320) et `director` (614). Un film coproduit aura par exemple « United States, India » comme pays. Tant que pandas compte « United States, India » comme un seul pays, l'analyse par pays est faussée. Dans `country` , 7 valeurs ont en plus une virgule de trop, au début (« , South Korea ») ou à la fin (« Poland, »). On s'en occupe en 2.6.

1.5 Bilan de l'état des lieux

La dernière colonne renvoie à la section où chaque problème est traité, et l'avant-dernière donne la correction finalement retenue.

Problème repéré	Colonne(s)	Lignes touchées	Correction retenue	Section
Espaces au début de la date	`date_added`	88	Retirer les espaces avant la conversion	2.1 / 2.3
Espaces insécables `\xa0`	`title` , `description`	5 + 9	Remplacer par une espace normale	2.1
Espaces de largeur nulle `\u200b`	`title` , `cast`	5 + 1	Supprimer	2.1
Doubles espaces	`title` , `cast` , `description`	3 + 7 + 6	Réduire à une seule espace	2.1
Durée rangée dans `rating` , `duration` vide	`rating` , `duration`	3	Déplacer dans `duration` , laisser `rating` vide	2.2
Date stockée en texte	`date_added`	8 797	Convertir en date, ajouter `annee_ajout`	2.3
Deux unités dans une colonne	`duration`	8 804	Séparer en `duree_min` et `nb_saisons`	2.4
Réalisateur manquant (surtout les séries)	`director`	2 634	Étiquette « Non renseigné », aucune valeur inventée	2.5
Pays et acteurs manquants	`country` , `cast`	831, 825	Étiquette « Non renseigné », pas de pays le plus fréquent	2.5
Quelques manquants	`date_added` , `rating`	10, 4	`date_added` laissée vide, `rating` en « Non renseigné »	2.5
Plusieurs valeurs par case	`country` , `listed_in` , `cast` , `director`	1 320, 6 787, 7 101, 614	Tables longues pays et genres	2.6
Virgule en trop au début ou à la fin	`country`	7	Retirer les pays vides	2.6
`NR` et `UR` = même sens	`rating`	80 + 3	Fusionner	2.7
`TV-Y7-FV` variante de `TV-Y7`	`rating`	6	Gardé tel quel : classification officielle	2.7
Même œuvre présente deux fois	`title`	4 paires sur 6	4 doublons supprimés, 2 paires gardées	2.7
Titres abîmés par Excel (trouvés en 2.7)	`title`	4	3 doublons supprimés, 1 titre corrigé	2.7
Ajouté avant l'année de sortie	`date_added` , `release_year`	14	Gardés et signalés par `ajout_avant_sortie`	2.7

2. Corrections justifiées

Une sous-partie par problème. Avant de supprimer quoi que ce soit : **mesurer la perte**.

2.1 Espaces superflus et texte à harmoniser · *Omotola*

Colonnes concernées : toutes les colonnes texte, en pratique `title` , `cast` , `description` et `date_added` (voir 1.4).

```
# On nettoie toutes les colonnes texte d'un coup, comme ça on attrape aussi ce qu'on n'aurait pas vu en 1.4
texte = df.select_dtypes(exclude="number").columns
avant = df[texte].copy()  # photo avant nettoyage, pour compter ce qui a changé

for col in texte:
```

```
    # L'ordre compte : strip() ne connaît pas \u200b et laisse les \xa0 du milieu,
    # donc on s'en occupe avant
    df[col] = (
        df[col]
        .str.replace("\u200b", "", regex=False)  # supprimé, pas remplacé par une espace
        .str.replace("\xa0", " ", regex=False)
        .str.replace(r" {2,}", " ", regex=True)  # 2 espaces ou plus deviennent 1
        .str.strip()
    )

# Piège : pour pandas deux cases vides ne sont pas "égales",
# sans le notna() tous les NaN compteraient comme modifiés
modifiees = (df[texte] != avant) & avant.notna()
print("Cases modifiées par colonne :")
print(modifiees.sum()[modifiees.sum() > 0], "\n")
print("Lignes touchées au total :", modifiees.any(axis=1).sum(), "\n")

# On revérifie qu'il ne reste plus rien
for nom, motif in [("espaces insécables", "\xa0"), ("espaces de largeur nulle", "\u200b"), ("doubles espaces",
"  ")]:
    reste = sum(df[c].str.contains(motif, regex=False, na=False).sum() for c in texte)
    print(f"{nom} restants : {reste}")
```

```
Cases modifiées par colonne :
title          13
cast            8
date_added     88
description    15
dtype: int64

Lignes touchées au total : 122

espaces insécables restants : 0
espaces de largeur nulle restants : 0
doubles espaces restants : 0
```

Décision : on a nettoyé les espaces dans toutes les colonnes texte, pas seulement dans celles repérées en 1.4. Les espaces de largeur nulle (le caractère invisible `\u200b`) sont supprimés, les espaces insécables (`\xa0`) deviennent des espaces normales, les doubles espaces sont ramenés à une seule, et on retire les espaces en trop au début et à la fin du texte.

Pourquoi ce choix plutôt qu'un autre : la méthode habituelle, `strip()` (qui enlève les espaces au début et à la fin d'un texte), ne suffisait pas. Elle laisse les espaces insécables qui sont au milieu d'un titre, comme dans « Legend of Exorcism », et elle ne reconnaît pas du tout l'espace de largeur nulle collé devant « Goli Soda 2 ». Ces caractères ne se voient pas, mais pour Python ils rendent différents deux textes identiques à l'écran. C'est ce qui cachait le doublon *Consequences* en 1.3. On remplace l'espace insécable par une espace normale au lieu de le supprimer, sinon « Akbar Birbal » serait devenu « AkbarBirbal ». Pour le reste, on ne touche pas au texte : majuscules et accents restent tels quels.

Lignes touchées : 122 lignes, dont 13 titres, 8 listes d'acteurs, 15 descriptions et 88 dates d'ajout. Après nettoyage, il ne reste plus aucun des trois caractères.

2.2 Valeurs décalées entre colonnes · *Omotola*

Si une valeur est dans la mauvaise colonne, peut-on la remettre à sa place sans rien inventer ?

```
# Les 3 spectacles de Louis C.K. vus en 1.4 : la durée est dans rating et duration est vide
decalees = df["rating"].str.contains("min", na=False)
print("Lignes décalées :", decalees.sum())

# D'abord on copie la durée, ensuite seulement on vide rating (dans l'autre sens on copierait du vide)
df.loc[decalees, "duration"] = df.loc[decalees, "rating"]
df.loc[decalees, "rating"] = np.nan  # on ne connaît pas leur vraie classification
```

```
print("rating contenant encore « min » :", df["rating"].str.contains("min", na=False).sum())
print("rating vides :", df["rating"].isna().sum(), "| duration vides :", df["duration"].isna().sum())
df.loc[decalees, ["show_id", "title", "rating", "duration"]]
```

```
Lignes décalées : 3
rating contenant encore « min » : 0
rating vides : 7 | duration vides : 0
```

	show_id	title	rating	duration
5541	s5542	Louis C.K. 2017	NaN	74 min
5794	s5795	Louis C.K.: Hilarious	NaN	84 min
5813	s5814	Louis C.K.: Live at the Comedy Store	NaN	66 min

Décision : pour les trois spectacles de Louis C.K., on a remis la durée (« 74 min », « 84 min », « 66 min ») de `rating` dans `duration`, qui était vide. Leur `rating` (classification par âge) est laissé vide.

Pourquoi ce choix plutôt qu'un autre : la valeur rangée dans `rating` a exactement la forme d'une durée de film, et la case `duration` de ces lignes était justement vide. On ne fait donc que la remettre à sa place, sans rien inventer. Pour la classification, c'est l'inverse : on ne sait pas laquelle Netflix a donnée à ces spectacles. Mettre la plus fréquente (`TV-MA` , réservée aux adultes) aurait été une supposition, alors on préfère un vide qu'on peut expliquer. Ces 3 vides s'ajoutent aux 4 `rating` déjà manquants, traités en 2.5. On fait cette correction avant la 2.4, sinon ces trois films n'auraient pas eu de durée.

Lignes touchées : 3 lignes. `rating` passe de 4 à 7 cases vides et `duration` n'en a plus aucune.

2.3 Date d'ajout (`date_added`) · *Omotola*

On convertit le texte en date. Combien de lignes échouent à la conversion, et pourquoi ?

```
# Les espaces du début ont déjà été retirés en 2.1, on peut convertir directement
vides_avant = df["date_added"].isna().sum()

# On donne le format des dates ("September 25, 2021") pour que pandas n'ait pas à deviner
df["date_added"] = pd.to_datetime(df["date_added"], format="%B %d, %Y", errors="coerce")
vides_apres = df["date_added"].isna().sum()

# Avec errors="coerce" une date ratée devient vide sans message d'erreur,
# d'où la comparaison des vides avant et après
print("Vides avant :", vides_avant, "| après :", vides_apres, "| échecs de conversion :", vides_apres -
vides_avant)
print("Type :", df["date_added"].dtype)
print("Période :", df["date_added"].min().date(), "à", df["date_added"].max().date())

# Int64 avec un grand I = entier qui accepte les vides (sans ça on aurait 2019.0)
df["annee_ajout"] = df["date_added"].dt.year.astype("Int64")
df["annee_ajout"].value_counts(dropna=False).sort_index()
```

```
Vides avant : 10 | après : 10 | échecs de conversion : 0
Type : datetime64[us]
Période : 2008-01-01 à 2021-09-25
```

```
annee_ajout
2008       2
2009       2
2010       1
2011      13
2012       3
2013      11
2014      24
2015      82
2016     429
2017    1188
2018    1649
2019    2016
2020    1879
2021    1498
<NA>      10
Name: count, dtype: Int64
```

Décision : on a transformé `date_added` en vraie date avec `pd.to_datetime` (la fonction de pandas qui lit un texte et le convertit en date), en lui indiquant comment les dates sont écrites (« September 25, 2021 »). On a aussi ajouté une colonne `annee_ajout` qui ne garde que l'année, pour pouvoir compter les ajouts par année.

Pourquoi ce choix plutôt qu'un autre : tant que la date reste du texte, on ne peut ni la trier dans l'ordre chronologique ni regrouper les titres par année. En précisant le format, on évite que pandas devine et confonde le jour et le mois. Avec l'option `errors="coerce"`, une date illisible devient vide au lieu de bloquer le notebook. C'est pour ça qu'on a compté les vides avant et après : leur nombre n'a pas bougé, donc aucune date n'a été perdue. Les 88 dates qui commençaient par une espace avaient déjà été corrigées en 2.1. Les 10 dates manquantes restent vides, elles sont discutées en 2.5. Dernier point : les données s'arrêtent au 25 septembre 2021, donc le chiffre de 2021 ne couvre que neuf mois et ne doit pas être comparé tel quel aux autres années.

Lignes touchées : les 8 797 dates renseignées sont converties sans aucun échec, les 10 vides restent vides.

2.4 Durée (`duration`) · *Omotola*

Minutes pour les films, saisons pour les séries : une seule colonne ne peut pas porter deux unités.

```python
# "90 min" ou "2 Seasons" : on récupère le nombre et l'unité séparément
parties = df["duration"].str.extract(r"^(\d+) (min|Seasons?)$")
nombre = pd.to_numeric(parties[0]).astype("Int64")
unite = parties[1]
# Si un format bizarre nous avait échappé, il apparaîtrait ici
print("Durées non reconnues :", (df["duration"].notna() & nombre.isna()).sum())

df["duree_min"] = nombre.where(unite == "min")
df["nb_saisons"] = nombre.where(unite.isin(["Season", "Seasons"]))

# Contrôle : un film ne doit pas avoir de saisons, ni une série de minutes
print(df.groupby("type")[["duree_min", "nb_saisons"]].count(), "\n")

# duration ne sert plus, tout est dans les deux nouvelles colonnes.
# NB : relancer cette cellule seule plante ensuite, il faut faire Redémarrer puis Exécuter tout
df = df.drop(columns="duration")

# 3 min et 312 min, ça nous a paru louche : on regarde de quels films il s'agit
films = df[df["type"] == "Movie"]
pd.concat([films.nsmallest(5, "duree_min"), films.nlargest(5, "duree_min")])[["title", "listed_in",
"duree_min"]]
```

```
Durées non reconnues : 0
```

```
         duree_min  nb_saisons
type
Movie         6131           0
TV Show          0        2676
```

	title	listed_in	duree_min
3777	Silent	Children & Family Movies, Sci-Fi & Fantasy	3
2713	Sol Levante	Action & Adventure, Anime Features, Internatio...	5
1484	Cops and Robbers	Dramas	8
1557	Canvas	Children & Family Movies, Dramas	9
3535	American Factory: A Conversation with the Obamas	Documentaries	10
4253	Black Mirror: Bandersnatch	Dramas, International Movies, Sci-Fi & Fantasy	312
717	Headspace: Unwind Your Mind	Documentaries	273
2491	The School of Mischief	Comedies, Dramas, International Movies	253
2487	No Longer kids	Comedies, Dramas, International Movies	237
2484	Lock Your Girls In	Comedies, International Movies, Romantic Movies	233

Décision : on a séparé `duration` en deux colonnes de nombres. `duree_min` contient la durée des films en minutes, `nb_saisons` le nombre de saisons des séries. Les deux sont de type `Int64` (des nombres entiers qui acceptent les cases vides). L'ancienne colonne `duration` est supprimée.

Pourquoi ce choix plutôt qu'un autre : avec « 90 min » et « 2 Seasons » dans la même colonne, aucun calcul n'est possible, et une moyenne qui mélange des minutes et des saisons ne voudrait rien dire. Avec deux colonnes, chacune n'a qu'une seule unité. Une série a donc une case vide dans `duree_min`, et c'est normal : la question ne se pose pas pour elle. On a vérifié avec `type` qu'aucun film n'a de saisons, qu'aucune série n'a de minutes, et qu'aucune durée n'a été oubliée au passage. On supprime `duration` parce que toute son information se retrouve dans les deux nouvelles colonnes : la garder aurait laissé deux versions de la même donnée dans le fichier final.

On s'est méfiés des durées extrêmes, de 3 à 312 minutes, mais elles sont toutes réelles. Les plus courtes sont des courts métrages comme *Silent* ou *Sol Levante*. *Black Mirror: Bandersnatch* est un film interactif (le spectateur choisit la suite de l'histoire), et ses 312 minutes additionnent toutes les fins possibles. *Headspace* est un programme de méditation lui aussi interactif. *The School of Mischief* (1973), *No Longer kids* (1979) et *Lock Your Girls In* (1982) sont des pièces de théâtre égyptiennes filmées, qui durent vraiment autour de quatre heures. On les garde toutes.

Lignes touchées : toutes les lignes, soit 6 131 films avec une durée en minutes et 2 676 séries avec un nombre de saisons. Les 3 durées remises en place en 2.2 sont incluses.

2.5 Réalisateurs, acteurs, pays manquants · *Amen*

Imputer, supprimer ou marquer ? Un vide peut être une vraie information (ex. pas de réalisateur crédité pour une série).

```
# Pour director, cast et country : combien de lignes perdrait-on si on supprimait les cases vides ?
cols = ["director", "cast", "country"]
vides = df[cols].isna().sum()
print(pd.DataFrame({"lignes_perdues": vides, "pct": (vides / len(df) * 100).round(1)}))

# Et si on supprimait toute ligne qui a au moins un de ces trois vides ?
une_colonne_vide = df[cols].isna().any(axis=1).sum()
print(f"\nAu moins un des trois vides : {une_colonne_vide} lignes ({une_colonne_vide / len(df):.1%})")

# Le vide dépend-il du type ? % de cases vides, pour les films puis pour les séries
print("\n% de cases vides selon le type :")
print((df[cols].isna().groupby(df["type"]).mean() * 100).round(1))

# On garde toutes les lignes et on écrit « Non renseigné » à la place du vide
touchees = df[cols].isna().sum()
df[cols] = df[cols].fillna("Non renseigné")
print("\nLignes touchées :", touchees.to_dict(), "| vides restants :", int(df[cols].isna().sum().sum()))
```

```
          lignes_perdues   pct
director            2634  29.9
cast                 825   9.4
country              831   9.4

Au moins un des trois vides : 3471 lignes (39.4%)

% de cases vides selon le type :
         director  cast  country
type
Movie         3.1   7.7      7.2
TV Show      91.4  13.1     14.6

Lignes touchées : {'director': 2634, 'cast': 825, 'country': 831} | vides restants : 0
```

Décision : on garde toutes les lignes. Dans `director`, `cast` et `country`, les cases vides sont remplacées par l'étiquette « Non renseigné ». Aucune valeur n'est inventée.

Pourquoi ce choix plutôt qu'un autre : on avait trois possibilités : supprimer, remplir avec une valeur probable, ou marquer le vide. Supprimer les lignes qui ont au moins un de ces trois vides ferait perdre 3 471 lignes, soit 39,4 % du catalogue, alors que leurs autres colonnes sont complètes. On jetterait presque deux titres sur cinq.

Remplir avec une valeur probable (on dit « imputer ») reviendrait à inventer de l'information. Mettre le pays le plus fréquent, les États-Unis, dans les 831 pays manquants gonflerait artificiellement ce pays et fausserait l'analyse par pays, qui est justement le sujet de l'exercice. Pour `director`, le vide concerne 91,4 % des séries (voir 1.2) : une série a souvent un réalisateur par épisode, donc l'information n'existe pas au niveau du titre, ce n'est pas un oubli. Pour `cast` et `country`, on ne peut pas savoir si l'information existe, et on ne la devine pas.

Restait à choisir entre une case vide et une étiquette. On a pris l'étiquette parce que pandas ignore les cases vides sans prévenir quand on compte (`value_counts()`) ou qu'on regroupe (`groupby()`). Avec « Non renseigné », le manque apparaît dans les résultats et personne ne l'oublie. La contrepartie, c'est que « Non renseigné » ressemble à un pays dans les comptages : c'est pour ça qu'on l'exclut des tables par pays en 2.6.

Lignes touchées : 2 634 pour `director`, 825 pour `cast` et 831 pour `country` (voir la sortie ci-dessus). Aucune ligne n'est supprimée.

```
# date_added (10 vides) et rating (vides restants après la correction 2.2)
print("date_added vides :", df["date_added"].isna().sum(), "| type de contenu :",
df.loc[df["date_added"].isna(), "type"].value_counts().to_dict())
print("rating vides     :", df["rating"].isna().sum(), "(4 d'origine + 3 durées décalées en 2.2)")

# Pour rating on écrit « Non renseigné », pour date_added on laisse la case vide
touchees_rating = int(df["rating"].isna().sum())
df["rating"] = df["rating"].fillna("Non renseigné")
print("\nLignes touchées : rating =", touchees_rating, "| date_added laissée vide :",
df["date_added"].isna().sum())
```

```
date_added vides : 10 | type de contenu : {'TV Show': 10}
rating vides     : 7 (4 d'origine + 3 durées décalées en 2.2)

Lignes touchées : rating = 7 | date_added laissée vide : 10
```

Décision : les 10 `date_added` vides restent vides. Les 7 `rating` vides deviennent « Non renseigné ».

Pourquoi ce choix plutôt qu'un autre : une date ne peut pas porter une étiquette, et en fabriquer une (une date moyenne, celle d'un titre voisin…) fausserait l'analyse par année. Supprimer ces 10 lignes (0,1 %) se défendrait aussi, mais on perdrait leur pays et leur genre, qui sont complets. On les garde donc : elles sont simplement ignorées quand on compte les ajouts par année.

Pour `rating`, on ne devine pas une classification. L'étiquette « Non renseigné » est volontairement différente de `NR` (*Not Rated*, « non classé ») : `NR` est une vraie valeur du fichier, qui dit que le titre n'a pas reçu de classification officielle, alors que « Non renseigné » veut dire que l'information manque.

Lignes touchées : 10 lignes de `date_added` laissées vides (ce sont toutes des séries) et 7 lignes de `rating`.

2.6 Colonnes à plusieurs valeurs (`country`, `listed_in`, `cast`, `director`) · *Amen*

Comment analyser « par pays » ou « par genre » quand une case contient plusieurs valeurs séparées par des virgules ?

```
# On découpe chaque case sur la virgule, on retire les morceaux vides (« Poland, » donne « Poland » et un
morceau vide),
# puis on recolle avec le même séparateur partout : une virgule et une espace
def normaliser(s):
    morceaux = s.str.split(",").map(lambda l: [x.strip() for x in l if x.strip()] if isinstance(l, list) else
l)
    return morceaux.map(lambda l: ", ".join(l) if isinstance(l, list) and l else np.nan)

for c in ["country", "listed_in", "cast", "director"]:
    avant = df[c].copy()
    df[c] = normaliser(df[c])
    print(f"{c:10} : {int((avant.notna() & (avant != df[c])).sum())} lignes modifiées")

# Les 7 pays avec une virgule en trop (repérés en 1.4), avant et après
mal_places = brut["country"].str.contains(r"^,|,$", na=False)
pd.DataFrame({"avant": brut.loc[mal_places, "country"], "après": df.loc[mal_places, "country"]})
```

```
country    : 7 lignes modifiées
listed_in  : 0 lignes modifiées
cast       : 0 lignes modifiées
director   : 0 lignes modifiées
```

	avant	après
193	, South Korea	South Korea
365	, France, Algeria	France, Algeria
1192	United Kingdom,	United Kingdom
2224	France, Belgium, Luxembourg, Cambodia,	France, Belgium, Luxembourg, Cambodia
4653	United States,	United States
5925	United Kingdom,	United Kingdom
7007	Poland,	Poland

Décision : dans `country`, `listed_in`, `cast` et `director`, on utilise partout le même séparateur (une virgule suivie d'une espace) et on retire les virgules en trop au début ou à la fin d'une case. Le fichier principal garde **une seule ligne par titre**.

Pourquoi ce choix plutôt qu'un autre : si on découpait les cases sans les nettoyer, « Poland, » donnerait un pays vide `""` qui s'ajouterait aux comptages. On ne sépare pas non plus les valeurs dans `df` lui-même : un titre à trois pays deviendrait trois lignes, et on compterait trois fois le même film. La séparation se fait dans des tables à part, à la cellule suivante.

Lignes touchées : 7 lignes pour `country`, ce sont les 7 virgules mal placées repérées en 1.4. Les trois autres colonnes avaient déjà un séparateur régulier (0 ligne modifiée).

```
# Table « longue » : une ligne par couple (titre, pays) et une ligne par couple (titre, genre).
# La colonne show_id permet de retrouver le titre, le type ou l'année en reliant la table à df (avec merge).
def table_longue(d, colonne, nom):
    t = d[["show_id", colonne]].dropna()
    t = t.assign(**{nom: t[colonne].str.split(", ")}).explode(nom)  # explode : une ligne par valeur de la
liste
    return t[["show_id", nom]].query(f"{nom} != 'Non renseigné'").reset_index(drop=True)

df_pays = table_longue(df, "country", "pays")
df_genres = table_longue(df, "listed_in", "genre")
print("table pays   :", df_pays.shape, "| table genres :", df_genres.shape)
```

```
# Contrôle : tous les titres qui ont un pays se retrouvent dans la table
print("titres avec pays :", df_pays["show_id"].nunique(), "| attendu :", int((df["country"] != "Non
renseigné").sum()))

# Pourquoi c'est nécessaire : compter les États-Unis avec et sans découpage
print("\nTitres dont country est exactement « United States » :", int((df["country"] == "United
States").sum()))
print("Titres qui ont les États-Unis parmi leurs pays          :", int((df_pays["pays"] == "United
States").sum()))

# Pourquoi on ne découpe pas cast : certains noms contiennent une virgule (« Sammy Davis, Jr. »)
morceaux = df["cast"].str.split(", ").explode()
print("\nMorceaux « Jr. » ou « Sr. » isolés dans cast :", int(morceaux.isin(["Jr.", "Sr.", "Jr",
"Sr"]).sum()))

# Les genres : combien de libellés différents, et les deux libellés très généraux
genres = df_genres["genre"].value_counts()
print("\ngenres différents :", len(genres), "| « Movies » :", genres.get("Movies", 0), "| « TV Shows » :",
genres.get("TV Shows", 0))
```

```
table pays   : (10012, 2) | table genres : (19323, 2)
titres avec pays : 7976 | attendu : 7976

Titres dont country est exactement « United States » : 2819
Titres qui ont les États-Unis parmi leurs pays          : 3690

Morceaux « Jr. » ou « Sr. » isolés dans cast : 2

genres différents : 42 | « Movies » : 57 | « TV Shows » : 16
```

Décision : on crée deux tables, `df_pays` (colonnes `show_id` et `pays`) et `df_genres` (colonnes `show_id` et `genre`), exportées en CSV à part en partie 4. `cast` et `director` restent en listes dans le fichier principal, et les libellés de genres sont gardés tels quels.

Pourquoi ce choix plutôt qu'un autre : une table « longue » a une ligne par couple (titre, pays). C'est plus propre que des colonnes `pays_1`, `pays_2` … : le nombre de pays varie d'un titre à l'autre, et ces colonnes seraient presque vides. La colonne `show_id` permet de retrouver le titre, son type ou son année en reliant la table à `df` (avec `merge`).

Le découpage change vraiment les résultats. Sans lui, « United States, India » compte comme un pays à part entière et les États-Unis sont sous-estimés : 2 819 titres ont les États-Unis comme seul pays, mais 3 690 les ont parmi leurs pays. « Non renseigné » est exclu de `df_pays` pour ne pas apparaître comme un pays.

On ne découpe ni `cast` ni `director`. Le sujet ne demande pas d'analyse par acteur ou par réalisateur, et couper `cast` sur la virgule casserait des noms comme « Sammy Davis, Jr. ». Pour `listed_in`, on garde les libellés de Netflix. `Dramas` (films) et `TV Dramas` (séries) sont deux catégories distinctes : les fusionner ferait perdre cette information, alors que `type` permet déjà de filtrer. Les libellés très généraux `Movies` (57 titres) et `TV Shows` (16) sont de vraies valeurs du fichier, on les garde et l'analyste peut les exclure.

Lignes touchées : aucune dans `df`. Les tables ont une ligne par couple (titre, pays) et (titre, genre), voir les dimensions affichées. Elles sont **reconstruites à l'export** (partie 4), après les suppressions de la 2.7, sinon un titre en double serait compté deux fois.

2.7 Autres problèmes trouvés par le binôme · *Amen*

Ce que la fiche n'annonçait pas : classifications équivalentes, doublons cachés, dates incohérentes, descriptions répétées, pays qui n'existent plus et titres abîmés par Excel.

```
# rating : NR et UR veulent dire la même chose ; faut-il aussi fusionner TV-Y7-FV avec TV-Y7 ?
print(df["rating"].value_counts().to_string())

n_ur = int((df["rating"] == "UR").sum())
```

```
df["rating"] = df["rating"].replace("UR", "NR")
print(f"\nLignes touchées : UR devient NR = {n_ur} | TV-Y7-FV laissé tel quel = {int((df['rating'] == 'TV-Y7-FV').sum())}")
```

```
rating
TV-MA            3207
TV-14            2160
TV-PG             863
R                 799
PG-13             490
TV-Y7             334
TV-Y              307
PG                287
TV-G              220
NR                 80
G                  41
Non renseigné       7
TV-Y7-FV            6
NC-17               3
UR                  3

Lignes touchées : UR devient NR = 3 | TV-Y7-FV laissé tel quel = 6
```

Décision : `UR` est fusionné dans `NR`. `TV-Y7-FV` reste séparé de `TV-Y7`.

Pourquoi ce choix plutôt qu'un autre : `NR` (*Not Rated*) et `UR` (*Unrated*) veulent tous les deux dire qu'aucune classification officielle n'a été donnée. Pour l'analyse ils sont équivalents, et les garder séparés ferait deux catégories pour la même chose. `TV-Y7-FV` est au contraire une classification officielle à part entière (enfants de plus de 7 ans, avec violence fantastique). Seulement 6 titres sont concernés, et `rating` ne fait pas partie des trois axes du sujet (pays, genre, année) : on évite de modifier une valeur officielle sans nécessité.

Lignes touchées : 3 lignes (`UR` devient `NR`). Les 6 `TV-Y7-FV` ne sont pas modifiés.

```
# Doublons cachés : même titre (majuscules ignorées). Maintenant que 2.1 a retiré les caractères invisibles,
# les deux « Consequences » sont reconnus comme identiques
doubles = df[df["title"].str.lower().duplicated(keep=False)].sort_values("title")
print("Paires de titres identiques à la casse près :", len(doubles) // 2)
doubles[["show_id", "type", "title", "director", "release_year", "date_added"]]
```

```
Paires de titres identiques à la casse près : 6
```

	show_id	type	title	director	release_year	date_added
3371	s3372	Movie	Consequences	Ozan Açıktan	2014	2019-10-25
6529	s6530	Movie	Consequences	Ozan Açıktan	2014	2019-10-25
5751	s5752	TV Show	DEATH NOTE	Non renseigné	2006	2016-10-15
5318	s5319	Movie	Death Note	Adam Wingard	2017	2017-08-25
6705	s6706	Movie	Esperando La Carroza	Alejandro Doria	1985	2018-07-15
303	s304	Movie	Esperando la carroza	Alejandro Doria	1985	2021-08-05
5033	s5034	Movie	FullMetal Alchemist	Fumihiko Sori	2017	2018-02-19
5095	s5096	TV Show	Fullmetal Alchemist	Non renseigné	2003	2018-01-01
7345	s7346	Movie	Love In A Puff	Pang Ho-cheung	2010	2018-08-01
159	s160	Movie	Love in a Puff	Pang Ho-cheung	2010	2021-09-01
8022	s8023	TV Show	Sin Senos sí Hay Paraíso	Non renseigné	2018	2019-01-11
1270	s1271	TV Show	Sin senos sí hay paraíso	Non renseigné	2018	2021-02-25

```
# On supprime seulement les vrais doublons : même titre + même type + même année de sortie.
# On trie par date d'ajout et on garde la dernière (la plus récente) de chaque groupe
ids_avant = set(df["show_id"])
n0 = len(df)
df = (df.assign(_titre=df["title"].str.lower())
        .sort_values("date_added", kind="stable", na_position="first")
        .drop_duplicates(["_titre", "type", "release_year"], keep="last")
        .drop(columns="_titre")
        .sort_index())
print(f"Lignes : {n0} avant, {len(df)} après ({n0 - len(df)} supprimées, {(n0 - len(df)) / n0:.2%})")
print("show_id supprimés :", sorted(ids_avant - set(df["show_id"])))
```

```
Lignes : 8807 avant, 8803 après (4 supprimées, 0.05%)
show_id supprimés : ['s3372', 's6706', 's7346', 's8023']
```

Décision : sur les 6 paires de titres identiques, on supprime les **4 vrais doublons** et on garde les 2 paires qui sont des œuvres différentes. Pour chaque doublon, on garde la ligne dont la date d'ajout est la plus récente.

Pourquoi ce choix plutôt qu'un autre : notre critère est le même titre (majuscules ignorées), le même `type` et la même `release_year`. *Death Note* et *FullMetal Alchemist* existent chacun en film et en série d'animation, avec un type et une année différents : ce sont des œuvres distinctes qui portent le même nom, on garde les deux.

Les 4 autres (*Consequences*, *Esperando la carroza*, *Love in a Puff*, *Sin senos sí hay paraíso*) ont la même année, la même description et le même réalisateur quand il est renseigné. Seuls changent le `show_id` et la date d'ajout : Netflix les a probablement retirés puis remis en ligne. Les compter deux fois gonflerait les comptes par pays, par genre et par année.

On garde la date la plus récente parce qu'elle correspond à la mise en ligne actuelle. Pour *Consequences*, les deux dates sont identiques, et on garde la dernière ligne du fichier. Avec l'autre choix, les comptes par pays et par genre seraient les mêmes, seule l'année d'ajout pourrait changer. Cette correction n'est possible qu'**après la 2.1** : tant que le caractère invisible collé à la fin d'un des deux *Consequences* n'était pas retiré, le doublon passait inaperçu.

Lignes touchées : 4 lignes supprimées sur 8 807 (0,05 %). Leurs `show_id` sont affichés ci-dessus.

```
# Titres ajoutés sur Netflix avant leur année de sortie : on les signale, on ne les corrige pas
# (le fillna(False) sert aux 10 dates d'ajout vides, pour lesquelles la comparaison n'a pas de réponse)
df["ajout_avant_sortie"] = (df["annee_ajout"] < df["release_year"]).fillna(False).astype(bool)
signales = df[df["ajout_avant_sortie"]]
print("Titres signalés :", len(signales), "|", signales["type"].value_counts().to_dict())

# Notre hypothèse (1.4) : pour une série, release_year serait l'année de la dernière saison.
# Si c'est vrai, ces séries doivent avoir plusieurs saisons
series = signales[signales["type"] == "TV Show"]
print(f"Séries signalées avec plus d'une saison : {int((series['nb_saisons'] > 1).sum())} sur {len(series)}")
signales[["title", "type", "release_year", "date_added", "nb_saisons"]]
```

```
Titres signalés : 14 | {'TV Show': 12, 'Movie': 2}
Séries signalées avec plus d'une saison : 8 sur 12
```

	title	type	release_year	date_added	nb_saisons
1551	Hilda	TV Show	2021	2020-12-14	2
1696	Polly Pocket	TV Show	2021	2020-11-15	2
2920	Love Is Blind	TV Show	2021	2020-02-13	1
3168	Fuller House	TV Show	2020	2019-12-06	5
3287	Maradona in Mexico	TV Show	2020	2019-11-13	1
3369	BoJack Horseman	TV Show	2020	2019-10-25	6
3433	The Hook Up Plan	TV Show	2020	2019-10-11	2
4844	Unbreakable Kimmy Schmidt	TV Show	2019	2018-05-30	4
4845	Arrested Development	TV Show	2019	2018-05-29	5
5394	Hans Teeuwen: Real Rancour	Movie	2018	2017-07-01	<NA>
5658	Sense8	TV Show	2018	2016-12-23	2
5677	Tokyo Trial	TV Show	2017	2016-12-13	1
7063	Incoming	Movie	2019	2018-10-26	<NA>
7112	Jack Taylor	TV Show	2016	2013-03-31	1

Décision : on garde ces 14 titres et on ajoute une colonne `ajout_avant_sortie` (vrai ou faux) pour les repérer. Rien n'est corrigé ni supprimé.

Pourquoi ce choix plutôt qu'un autre : notre hypothèse de la 1.4 était que, pour une série, `release_year` donne l'année de la dernière saison. Si cette saison est sortie après l'arrivée de la série sur Netflix, `release_year` est forcément plus récent que la date d'ajout. L'hypothèse tient pour **8 des 12 séries**, qui ont plusieurs saisons (*BoJack Horseman*, *Fuller House*, *Sense8*…). Elle n'explique **pas** les 4 séries d'une seule saison ni les 2 films, et le fichier seul ne permet pas de savoir quelle date est la bonne.

Corriger reviendrait donc à deviner, et supprimer ferait perdre 14 titres dont le pays et le genre sont exacts. La nouvelle colonne permet d'exclure ces lignes quand on étudie l'évolution dans le temps.

Lignes touchées : 14 lignes signalées, aucune modifiée ni supprimée.

```
# Deux constats, sans modification : descriptions partagées et pays qui n'existent plus
partagees = df[df["description"].duplicated(keep=False)].sort_values("description")
print("Titres dont la description est partagée avec un autre titre :", len(partagees))
display(partagees[["title", "type"]].head(6))

pays = df["country"].str.split(", ").explode().value_counts()
print()
for p in ["Germany", "West Germany", "East Germany", "Soviet Union"]:
    print(f"{p:13}: {pays.get(p, 0)} titres")
```

```
Titres dont la description est partagée avec un autre titre : 53
```

	title	type
78	Tughlaq Durbar	Movie
79	Tughlaq Durbar (Telugu)	Movie
2969	Together For Eternity	Movie
7022	Hum Saath-Saath Hain	Movie
3492	Oh! Baby (Malayalam)	Movie
3493	Oh! Baby (Tamil)	Movie

```
Germany       : 226 titres
West Germany  : 5 titres
East Germany  : 1 titres
Soviet Union  : 3 titres
```

Décision : aucune modification dans cette cellule. On fait deux constats et on les documente.

Pourquoi ce choix plutôt qu'un autre : la plupart des titres qui partagent une description sont des versions d'un même film dans une autre langue, avec un titre et un `show_id` différents : *Tughlaq Durbar* et *Tughlaq Durbar (Telugu)*, *Oh! Baby* et ses versions tamoule et malayalam, *Solo: A Star Wars Story* et sa version espagnole. Netflix les présente comme des entrées distinctes, ce ne sont pas des doublons à supprimer.

En les passant toutes en revue, on a trouvé deux autres cas. D'abord des titres abîmés par Excel, comme « 22-Jul » pour « 22 July », traités à la cellule suivante. Ensuite quelques titres écrits de deux façons, comme *Chashme Baddoor* et *Chashme Buddoor*, ou deux orthographes d'un même film de *Naruto Shippuden* et d'*InuYasha*. Ce sont des doublons probables, mais le titre diffère et parfois aussi la distribution ou la classification : rien dans le fichier ne permet de trancher, et ce pourrait être une autre version du film. On les garde et on le signale.

Pour les pays, `West Germany`, `East Germany` et `Soviet Union` désignent le vrai pays de production de titres anciens. Les fusionner avec `Germany` ou avec la Russie changerait le sens de la donnée. On les garde, en sachant qu'un total « Allemagne » qui ne compte que `Germany` est légèrement incomplet.

Lignes touchées : 0 (constats seulement, les effectifs sont affichés ci-dessus).

```
# En lisant les descriptions partagées, on est tombés sur « 22 July » et « 22-Jul » :
# Excel a pris certains titres pour des dates et les a réécrits. On cherche tous les titres de ce genre
abimes = df["title"].str.match(r"^\d{1,2}-[A-Z][a-z]{2}$|^[A-Z][a-z]{2}-\d{2}$")
print("Titres abîmés par Excel :", df.loc[abimes, "title"].tolist())

# Un « jumeau », c'est une autre ligne identique sur toutes les colonnes sauf show_id et title
autres_colonnes = [c for c in df.columns if c not in ["show_id", "title"]]
a_un_jumeau = df.duplicated(autres_colonnes, keep=False)
display(df[a_un_jumeau & df["description"].isin(df.loc[abimes, "description"])]
        .sort_values("description")[["show_id", "title", "director", "release_year", "date_added"]])

# Il faut les deux conditions : les versions doublées (Boomika, Boomika (Hindi)...) ont elles aussi un jumeau
a_supprimer = df.loc[abimes & a_un_jumeau, "show_id"].tolist()
df = df[~df["show_id"].isin(a_supprimer)]
```

```
# « Oct-01 » n'a pas de jumeau : c'est le film nigérian October 1, on lui rend son titre
df.loc[df["title"] == "Oct-01", "title"] = "October 1"
print("show_id supprimés :", a_supprimer, "| lignes restantes :", len(df))
```

```
Titres abîmés par Excel : ['Oct-01', 'Feb-09', '22-Jul', '15-Aug']
```

	show_id	title	director	release_year	date_added
4522	s4523	22 July	Paul Greengrass	2018	2018-10-10
5965	s5966	22-Jul	Paul Greengrass	2018	2018-10-10
3996	s3997	February 9	Non renseigné	2018	2019-03-20
5964	s5965	Feb-09	Non renseigné	2018	2019-03-20
3962	s3963	15 August	Swapnaneel Jayakar	2019	2019-03-29
5966	s5967	15-Aug	Swapnaneel Jayakar	2019	2019-03-29

```
show_id supprimés : ['s5965', 's5966', 's5967'] | lignes restantes : 8800
```

Décision : on supprime les 3 titres abîmés qui ont un jumeau (`Feb-09` , `22-Jul` et `15-Aug`) et on rend son vrai titre au quatrième : « Oct-01 » redevient « October 1 ».

Pourquoi ce choix plutôt qu'un autre : « 22 July » et « 22-Jul » ont exactement la même description, le même réalisateur, la même date d'ajout, la même durée. Le titre « 22-Jul » a été abîmé par Excel : quand on ouvre un CSV dans Excel, il prend « 22 July » pour une date et la réécrit à sa façon. Les quatre titres concernés se suivent dans le fichier (s5964 à s5967), ce qui va dans le même sens.

Pour trois d'entre eux, une autre ligne au titre correct est identique sur toutes les autres colonnes : ce sont des doublons, qu'on supprime comme les précédents. On garde la ligne au bon titre, puisque l'autre est abîmée. Le quatrième, « Oct-01 », n'a pas de jumeau : c'est le seul exemplaire du film nigérian *October 1* (2014), dont la description parle de l'indépendance du Nigeria, proclamée le 1er octobre 1960. On ne le supprime pas, on corrige seulement son titre.

On ne pouvait pas se contenter de chercher les lignes identiques à part le titre. Les versions dans d'autres langues (*Boomika* et *Boomika (Hindi)*, par exemple) sont elles aussi identiques sur tout le reste, et ce ne sont pas des doublons. C'est pour ça que le code exige les deux conditions : un titre abîmé **et** un jumeau.

Lignes touchées : 3 lignes supprimées (s5965, s5966, s5967) et 1 titre corrigé (s5964). Le fichier passe de 8 803 à 8 800 lignes.

3. État final

On recalcule les mêmes indicateurs qu'en partie 1, cette fois sur `df` , et on les met à côté de ceux de `brut` . Tous les chiffres viennent du code, rien n'est recopié à la main.

3.1 Dimensions et types

```
print(f"Avant : {brut.shape[0]} lignes, {brut.shape[1]} colonnes")
print(f"Après : {df.shape[0]} lignes, {df.shape[1]} colonnes")
perdues = len(brut) - len(df)
print(f"Lignes supprimées : {perdues} ({perdues / len(brut):.2%})")

# Les colonnes de brut d'abord, puis celles qu'on a créées. On s'en ressert dans les tableaux suivants
toutes = list(brut.columns) + [c for c in df.columns if c not in brut.columns]

# Même tableau qu'en 1.1, mais côte à côte. Une case vide veut dire que la colonne n'existe pas de ce côté-là
pd.DataFrame({
    "type_avant": brut.dtypes.astype(str),
    "type_apres": df.dtypes.astype(str),
    "valeurs_avant": brut.nunique(),
    "valeurs_apres": df.nunique(),
}).reindex(toutes).astype({"valeurs_avant": "Int64", "valeurs_apres": "Int64"})
```

```
Avant : 8807 lignes, 12 colonnes
Après : 8800 lignes, 15 colonnes
Lignes supprimées : 7 (0.08%)
```

	type_avant	type_apres	valeurs_avant	valeurs_apres
show_id	str	str	8807	8800
type	str	str	2	2
title	str	str	8807	8800
director	str	str	4528	4529
cast	str	str	7692	7691
country	str	str	748	744
date_added	str	datetime64[us]	1767	1714
release_year	int64	int64	74	74
rating	str	str	17	14
duration	str	NaN	220	<NA>
listed_in	str	str	514	514
description	str	str	8775	8774
annee_ajout	NaN	Int64	<NA>	14
duree_min	NaN	Int64	<NA>	205
nb_saisons	NaN	Int64	<NA>	15
ajout_avant_sortie	NaN	bool	<NA>	2

Constats

Le fichier passe de 8 807 à 8 800 lignes. On n'a supprimé que 7 lignes, soit 0,08 % du catalogue, et ce sont toutes des doublons. Il gagne trois colonnes : on est partis de 12, on a retiré `duration`, et on a ajouté `annee_ajout`, `duree_min`, `nb_saisons` et `ajout_avant_sortie` (voir 2.3, 2.4 et 2.7).

Les types correspondent maintenant au contenu. `date_added` est une vraie date (`datetime64`), et les durées, les saisons et l'année d'ajout sont des nombres entiers (`Int64`, le type entier qui accepte les cases vides). `show_id` et `title` ont autant de valeurs différentes que de lignes, 8 800 chacun : plus aucun titre n'apparaît deux fois à l'identique.

`rating` passe de 17 à 14 valeurs différentes : les 3 durées sont parties, `UR` a rejoint `NR`, et « Non renseigné » est arrivé.

3.2 Valeurs manquantes

```python
# On reprend la fonction manquants de la 1.2, sur brut puis sur df.
# Piège : un vide remplacé par « Non renseigné » n'est plus compté comme vide, donc on le compte à part
etiquette = (df[["director", "cast", "country", "rating"]] == "Non renseigné").sum()
print("Cases marquées « Non renseigné » :", int(etiquette.sum()))

bilan_vides = pd.DataFrame({
    "vides_avant": manquants(brut)["manquants"],
    "vides_apres": manquants(df)["manquants"],
    "non_renseigne": etiquette,
}).reindex(toutes).astype("Int64")
bilan_vides
```

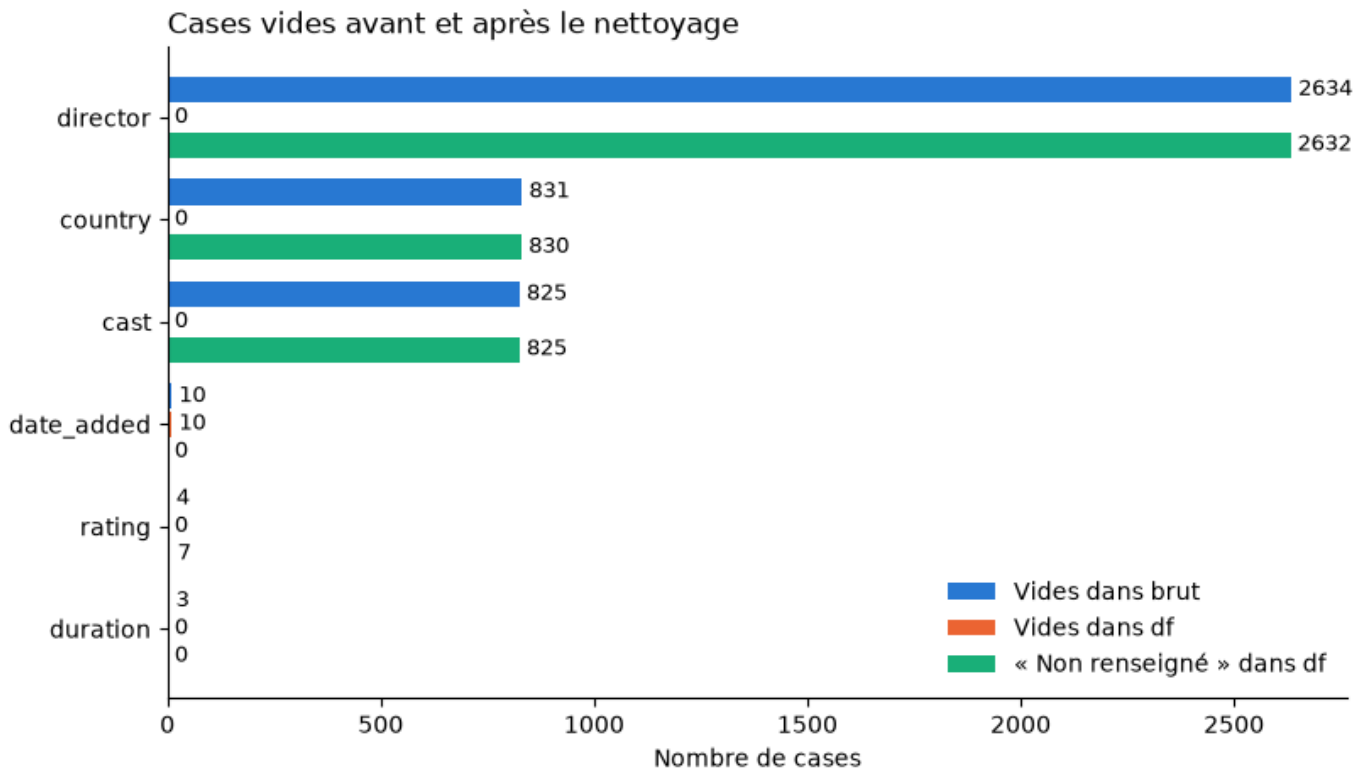
```
Cases marquées « Non renseigné » : 4294
```

	vides_avant	vides_apres	non_renseigne
show_id	0	0	<NA>
type	0	0	<NA>
title	0	0	<NA>
director	2634	0	2632
cast	825	0	825
country	831	0	830
date_added	10	10	<NA>
release_year	0	0	<NA>
rating	4	0	7
duration	3	<NA>	<NA>
listed_in	0	0	<NA>
description	0	0	<NA>
annee_ajout	<NA>	10	<NA>
duree_min	<NA>	2674	<NA>
nb_saisons	<NA>	6126	<NA>
ajout_avant_sortie	<NA>	0	<NA>

```
# Le même tableau en image, pour les six colonnes qui avaient des cases vides au départ
cols_vides = ["director", "country", "cast", "date_added", "rating", "duration"]
donnees = bilan_vides.loc[cols_vides].fillna(0).astype(int)
series = [
    ("vides_avant", "Vides dans brut", "#2a78d6"),
    ("vides_apres", "Vides dans df", "#eb6834"),
    ("non_renseigne", "« Non renseigné » dans df", "#1baf7a"),
]

fig, ax = plt.subplots(figsize=(9, 5))
y = np.arange(len(cols_vides))
for i, (col, nom, couleur) in enumerate(series):
    # Chaque série est décalée un peu vers le haut ou vers le bas pour que les barres ne se superposent pas
    barres = ax.barh(y + (i - 1) * 0.27, donnees[col], height=0.25, color=couleur, label=nom)
    ax.bar_label(barres, padding=3, fontsize=9)

ax.set_yticks(y, cols_vides)
ax.invert_yaxis()  # sinon director se retrouve en bas
ax.set_xlabel("Nombre de cases")
ax.set_title("Cases vides avant et après le nettoyage", loc="left")
ax.spines[["top", "right"]].set_visible(False)
ax.legend(frameon=False, loc="lower right")
plt.show()
```



Constats

Il faut lire ce tableau avec précaution. Les vides de `director`, `cast`, `country` et `rating` tombent à 0, mais on n'a retrouvé aucune information : 4 294 cases portent maintenant l'étiquette « Non renseigné ». Le manque est toujours là, il est simplement visible dans les comptages (voir 2.5). Le graphique le montre bien : la barre verte est presque aussi longue que la barre bleue.

Les petites différences ont une explication. `director` passe de 2 634 vides à 2 632 étiquettes et `country` de 831 à 830, parce que des lignes supprimées en 2.7 faisaient partie de ces vides. `rating` passe au contraire de 4 à 7, à cause des 3 spectacles de Louis C.K. de la 2.2. Les 3 vides de `duration` ont disparu sans étiquette, puisqu'on a retrouvé ces durées dans `rating`. C'est le seul cas où une information manquante a vraiment été récupérée, et la colonne `duration` n'existe plus aujourd'hui, ce qui explique son 0 dans le graphique.

Les 10 dates d'ajout vides restent vides, et `annee_ajout` hérite de ces 10 cases (voir 2.5). Enfin, `duree_min` et `nb_saisons` ont beaucoup de vides, 2 674 et 6 126, et c'est normal : une série n'a pas de durée en minutes, un film n'a pas de saisons. Leur total fait bien 8 800.

3.3 Doublons et problèmes repérés

```
# Les contrôles de la partie 1, regroupés dans une fonction pour les refaire à l'identique sur brut et sur df
def indicateurs(d):
    """Compte, dans le tableau d, les problèmes repérés en partie 1."""
    texte = d.select_dtypes(include="str").columns

    def cases_avec(motif):
        return sum(d[c].str.contains(motif, regex=False, na=False).sum() for c in texte)

    return {
        "Doublons stricts": d.duplicated().sum(),
        "show_id en double": d["show_id"].duplicated().sum(),
        "Titres en double (casse et espaces ignorés)": d["title"].str.lower().str.strip().duplicated().sum(),
        "Textes avec une espace au début ou à la fin": sum((d[c].notna() & (d[c].str.strip() != d[c])).sum()
for c in texte),
        "Textes avec un espace insécable": cases_avec("\xa0"),
        "Textes avec un espace de largeur nulle": cases_avec("\u200b"),
        "Textes avec un double espace": cases_avec("  "),
```

```
        "Durées rangées dans rating": d["rating"].str.contains("min", na=False).sum(),
        "Classifications UR": (d["rating"] == "UR").sum(),
        "Pays avec une virgule en trop": d["country"].str.contains(r"^,|,$", na=False).sum(),
        "Titres abîmés par Excel": d["title"].str.match(r"^\d{1,2}-[A-Z][a-z]{2}$|^[A-Z][a-z]{2}-
\d{2}$").sum(),
    }

pd.DataFrame({"avant": indicateurs(brut), "après": indicateurs(df)})
```

	avant	après
Doublons stricts	0	0
show_id en double	0	0
Titres en double (casse et espaces ignorés)	6	2
Textes avec une espace au début ou à la fin	89	0
Textes avec un espace insécable	14	0
Textes avec un espace de largeur nulle	6	0
Textes avec un double espace	16	0
Durées rangées dans rating	3	0
Classifications UR	3	0
Pays avec une virgule en trop	7	0
Titres abîmés par Excel	4	0

Constats

Tous les problèmes repérés en partie 1 sont à 0, sauf un, et c'est voulu. Il reste 2 titres en double une fois les majuscules ignorées : *Death Note* et *Fullmetal Alchemist*, qui existent chacun en film et en série. Ce sont des œuvres différentes qui portent le même nom, on les a gardées exprès (voir 2.7).

3.4 Le fichier répond-il au sujet ?

Le but de l'exercice est d'analyser les contenus par pays, par genre et par année. On vérifie que c'est possible maintenant, et on regarde ce que le nettoyage change aux résultats.

```
# Les tables de la 2.6 datent d'avant les suppressions de la 2.7 : on les refait sur le df final
df_pays = table_longue(df, "country", "pays")
df_genres = table_longue(df, "listed_in", "genre")

# Le même classement, sur le fichier d'origine puis sur les tables longues
print("Top 5 des pays dans brut :")
print(brut["country"].value_counts().head(5).to_string(), "\n")
print("Top 5 des pays dans df_pays :")
print(df_pays["pays"].value_counts().head(5).to_string(), "\n")

print("Top 5 des genres dans brut :")
print(brut["listed_in"].value_counts().head(5).to_string(), "\n")
print("Top 5 des genres dans df_genres :")
print(df_genres["genre"].value_counts().head(5).to_string(), "\n")

print("Titres ajoutés par année (les 6 dernières) :")
print(df["annee_ajout"].value_counts().sort_index().tail(6).to_string())
```

```
Top 5 des pays dans brut :
country
United States     2818
India              972
United Kingdom     419
Japan              245
South Korea        199

Top 5 des pays dans df_pays :
pays
United States     3688
India             1045
United Kingdom     806
Canada             445
France             393

Top 5 des genres dans brut :
listed_in
Dramas, International Movies                        362
Documentaries                                       359
Stand-Up Comedy                                     334
Comedies, Dramas, International Movies              274
Dramas, Independent Movies, International Movies    252

Top 5 des genres dans df_genres :
genre
International Movies      2749
Dramas                    2423
Comedies                  1671
International TV Shows    1349
Documentaries              869

Titres ajoutés par année (les 6 dernières) :
annee_ajout
2016     429
2017    1188
2018    1646
2019    2012
2020    1879
2021    1498
```

Constats

Les trois analyses fonctionnent, et le nettoyage change vraiment les réponses. Sur le fichier d'origine, les États-Unis ont 2 818 titres. Avec la table `df_pays`, ils en ont 3 688, parce qu'on compte aussi les coproductions. Le classement bouge : le Royaume-Uni double presque (de 419 à 806), le Canada et la France entrent dans le top 5, le Japon et la Corée du Sud en sortent.

Pour les genres, l'écart est encore plus net. Sur le fichier d'origine, le « genre » le plus fréquent est une combinaison, « Dramas, International Movies », qui ne veut rien dire pour une analyse. Avec `df_genres`, on obtient de vrais genres : *International Movies* (2 749 titres), *Dramas* (2 423) et *Comedies* (1 671).

Par année, les ajouts montent jusqu'à 2 012 titres en 2019. Attention à ne pas conclure à une baisse en 2021 : les données s'arrêtent au 25 septembre (voir 2.3).

3.5 Bilan du nettoyage

Le fichier d'origine comptait 8 807 lignes et 12 colonnes. Le fichier propre en compte 8 800 et 15. On a supprimé très peu de lignes, 7 doublons, et on n'a inventé aucune valeur. Chaque correction est expliquée dans sa cellule « Décision », avec le nombre de lignes touchées.

Le fichier n'est pas parfait pour autant, et il vaut mieux le dire. Il reste 4 294 cases « Non renseigné » et 10 dates d'ajout vides. 14 titres sont signalés par `ajout_avant_sortie` parce que leurs dates ne collent pas. Quelques

doublons probables sont restés, parce que leur titre est écrit de deux façons et que rien dans le fichier ne permet de trancher (voir 2.7). Enfin, 2021 est une année incomplète.

Avec plus de temps, on croiserait le catalogue avec une base extérieure comme IMDb. Ça permettrait de retrouver des pays et des réalisateurs manquants, de vérifier les doublons probables, et de connaître la vraie date de sortie des 14 titres signalés.

4. Export

On enregistre le fichier nettoyé et les deux tables longues dans `data/processed/`, puis on relit le fichier pour vérifier que rien ne s'est perdu en route.

```
PROPRE.parent.mkdir(parents=True, exist_ok=True)   # crée data/processed s'il n'existe pas

df.to_csv(PROPRE, index=False)
df_pays.to_csv(PROPRE.parent / "netflix_pays.csv", index=False)
df_genres.to_csv(PROPRE.parent / "netflix_genres.csv", index=False)
print("Fichier propre :", df.shape, "| pays :", df_pays.shape, "| genres :", df_genres.shape)

# Un CSV ne garde pas les types : il faut redire à pandas où sont les dates et les entiers qui ont des vides
entiers = {"annee_ajout": "Int64", "duree_min": "Int64", "nb_saisons": "Int64"}
relu = pd.read_csv(PROPRE, parse_dates=["date_added"], dtype=entiers)

# df a gardé ses numéros de ligne d'origine, avec des trous après les suppressions, d'où le reset_index
print("Le fichier relu est identique à df :", relu.equals(df.reset_index(drop=True)), "\n")
relu.info()
```

```
Fichier propre : (8800, 15) | pays : (10003, 2) | genres : (19304, 2)
Le fichier relu est identique à df : True

<class 'pandas.DataFrame'>
RangeIndex: 8800 entries, 0 to 8799
Data columns (total 15 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   show_id             8800 non-null   str
 1   type                8800 non-null   str
 2   title               8800 non-null   str
 3   director            8800 non-null   str
 4   cast                8800 non-null   str
 5   country             8800 non-null   str
 6   date_added          8790 non-null   datetime64[us]
 7   release_year        8800 non-null   int64
 8   rating              8800 non-null   str
 9   listed_in           8800 non-null   str
 10  description         8800 non-null   str
 11  annee_ajout         8790 non-null   Int64
 12  duree_min           6126 non-null   Int64
 13  nb_saisons          2674 non-null   Int64
 14  ajout_avant_sortie  8800 non-null   bool
dtypes: Int64(3), bool(1), datetime64[us](1), int64(1), str(9)
memory usage: 3.9 MB
```

Décision : on exporte trois fichiers en CSV : `netflix_titles_propre.csv` (une ligne par titre), `netflix_pays.csv` et `netflix_genres.csv` (les tables longues). Pour relire le fichier propre, on précise les colonnes de dates et d'entiers.

Pourquoi ce choix plutôt qu'un autre : un CSV est un simple fichier texte, il ne garde pas les types. Sans précision, `date_added` reviendrait en texte, et `annee_ajout`, `duree_min` et `nb_saisons` reviendraient en nombres décimaux (2019.0) à cause de leurs cases vides. En indiquant les types à la relecture, on retrouve exactement `df` : `equals` (qui compare deux tableaux case par case) le confirme.

Il ne faut pas ouvrir ces CSV dans Excel pour les réenregistrer. C'est comme ça que « 22 July » est devenu « 22-Jul » dans le fichier d'origine (voir 2.7).

Lignes touchées : 8 800 lignes dans le fichier propre, 10 003 dans la table des pays et 19 304 dans celle des genres.